



TÓPICOS EM INFORMÁTICA

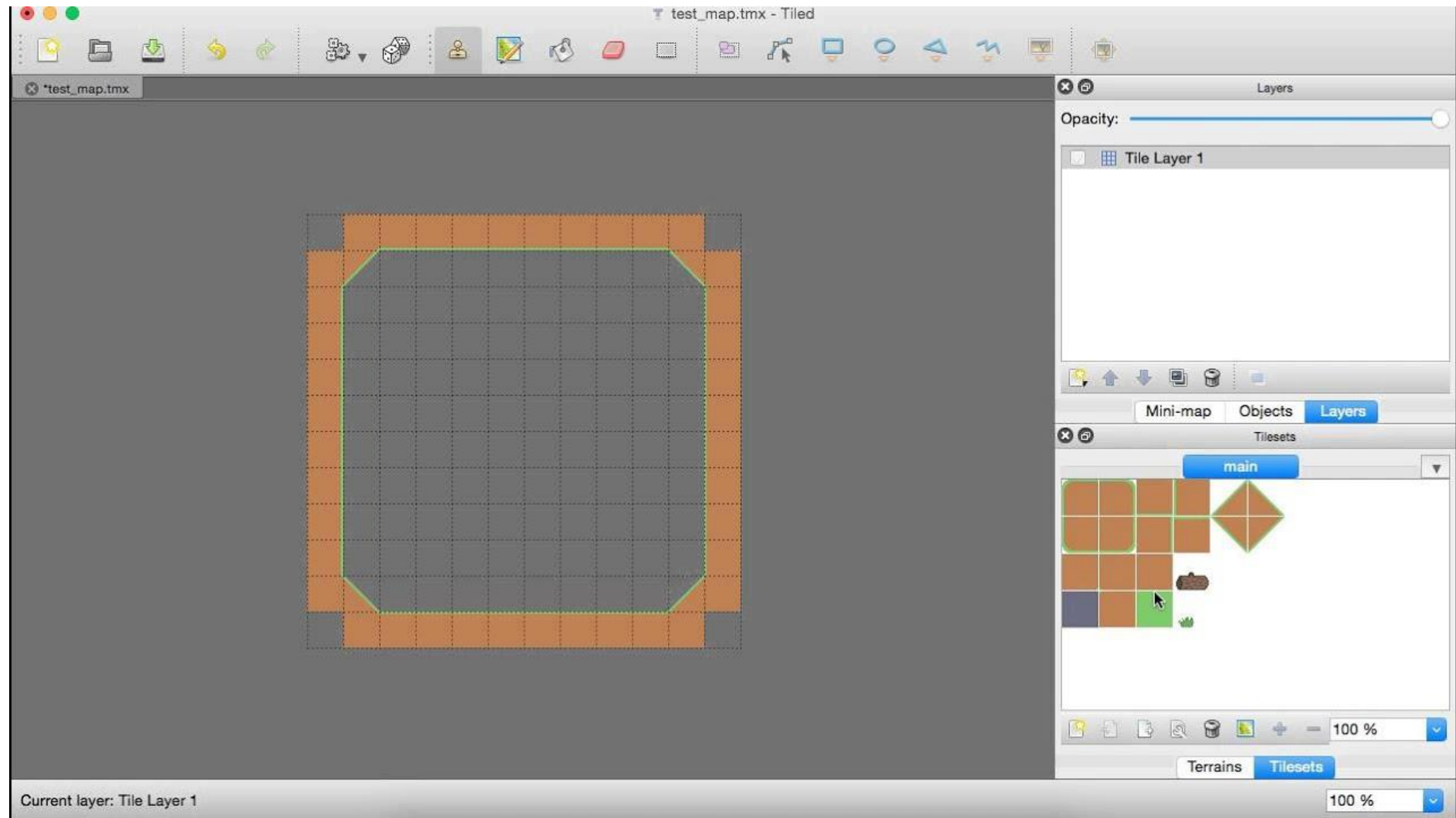
*Tile maps, bitmap font, ninepatches,
hitboxes, partículas 2d*



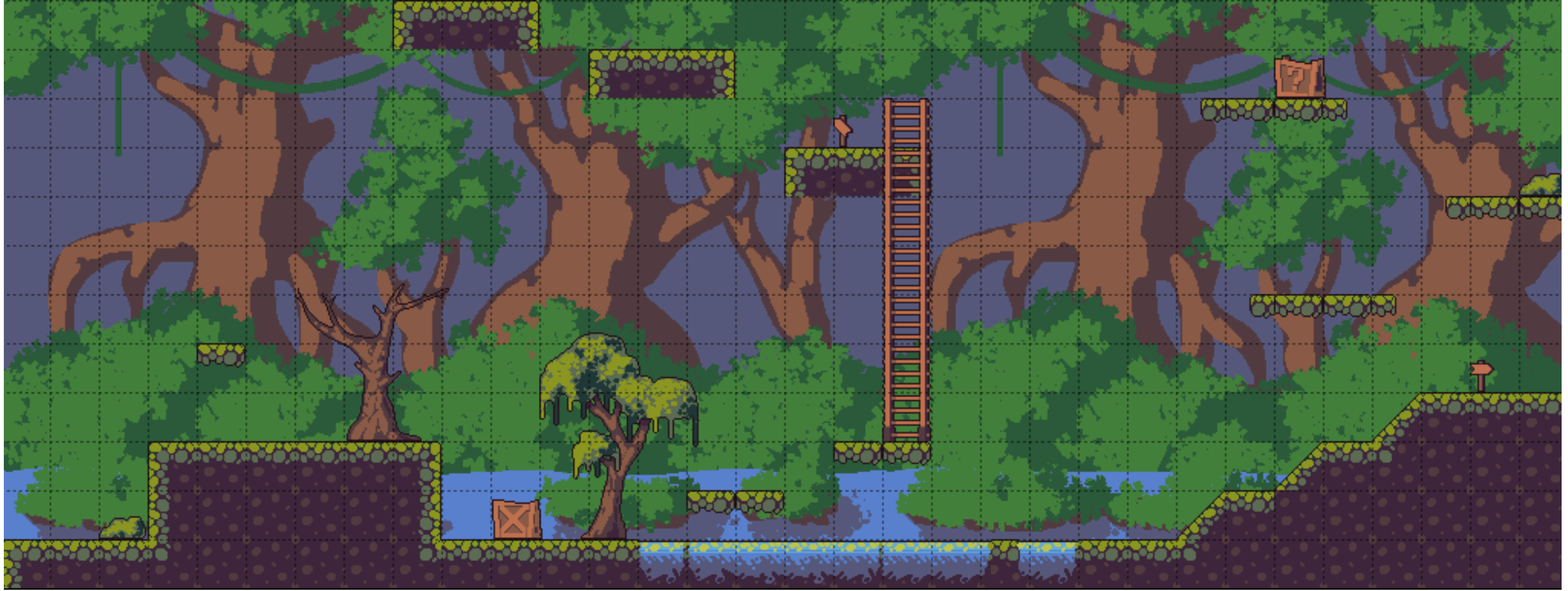
TILE MAPS

- A libgdx possui uma API **genérica** de mapas.
 - A API é baseada em **camadas de mapas**.
 - Possui suporte à tiled maps (**mapas com tiles – quadradinhos**).
- Aqui nesta disciplina, nós vamos focar em ver apenas como **importar os mapas de um editor chamado Tiled**.
 - Vocês podem acessar o seguinte link <https://www.mapeditor.org/> para baixar o programa.
- Utilizar o **Tiled** é **bastante simples**. Basta iniciar um novo mapa, carregar a textura e começar a desenhar.
 - Como previamente informado, é possível trabalhar com **várias camadas**.

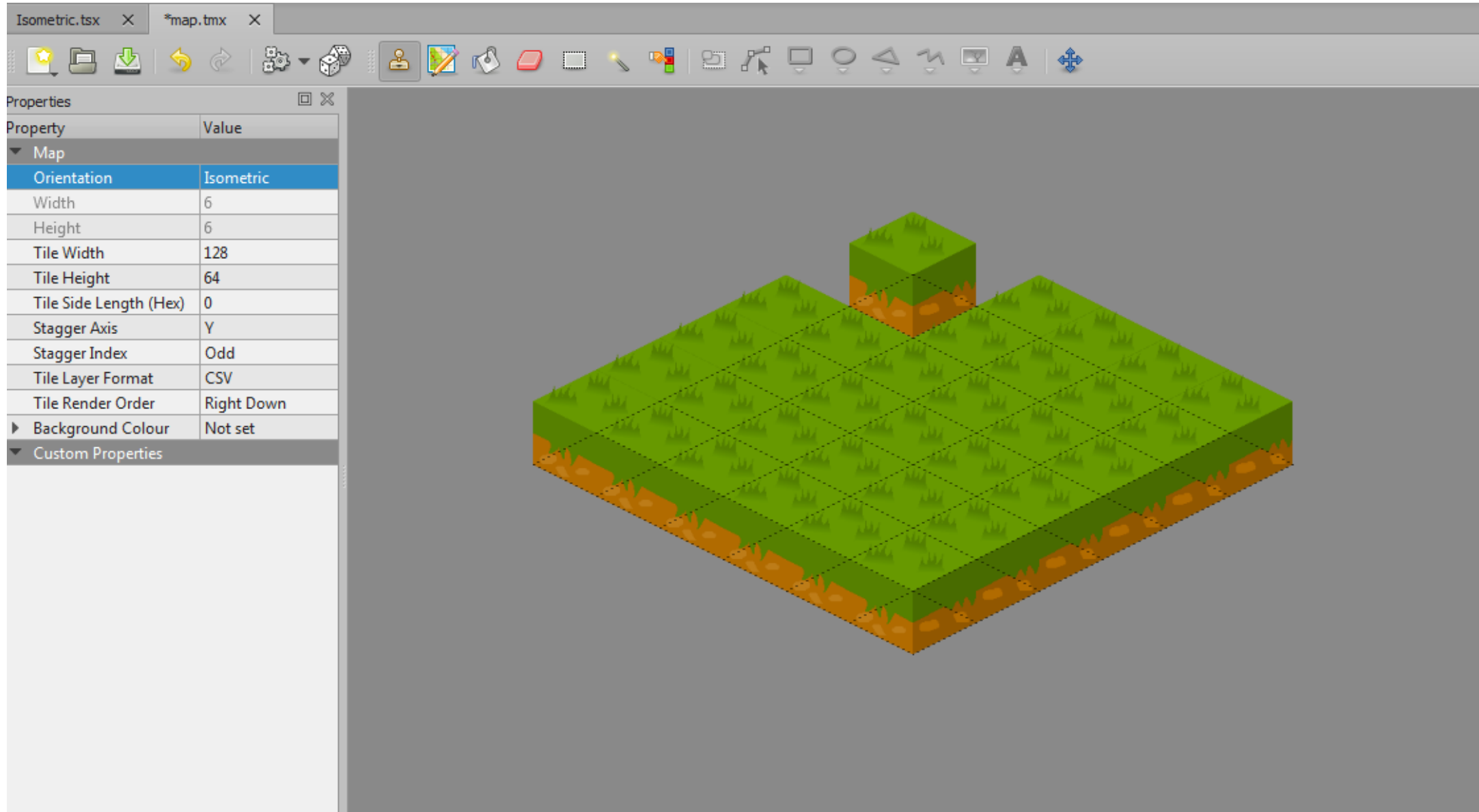
TILED



TILED



TILED





TILED

- Uma vez terminado, basta **exportar o mapa** e salvá-lo como .tmx. Para importar na libgdx você precisa declarar:

```
OrthogonalTiledMapRenderer tmr;  
TiledMap map;
```

- Depois, inicializar:

```
map = new TmxMapLoader().load("local/mapa.tmx");  
tmr = new OrthogonalTiledMapRenderer(map);
```

- Finalmente, no render (**não esqueça do `dispose()` também**):

```
tmr.setView(camera); //move o mapa de acordo com a câmera  
tmr.render(); //renderiza (desenha) o mapa
```



EXERCÍCIO

- Monte um mapa usando o Tiled e:
 - Insira no seu jogo,
 - Atualize a câmera de acordo com o movimento do personagem,
 - Insira algum tipo de colisão com o mapa (pode ser manualmente ou com o Box2D).



BITMAP FONTS

- Vocês devem ter reparado até agora com a experiência da libgdx que tudo que é “desenhado” na tela por controladores gráficos como o OpenGL **são texturas** (e a libgdx é **apenas um link ou framework que trabalha em cima do OpenGL**).
- Dito isto, como seria possível escrever texto nesse contexto?



BITMAP FONTS

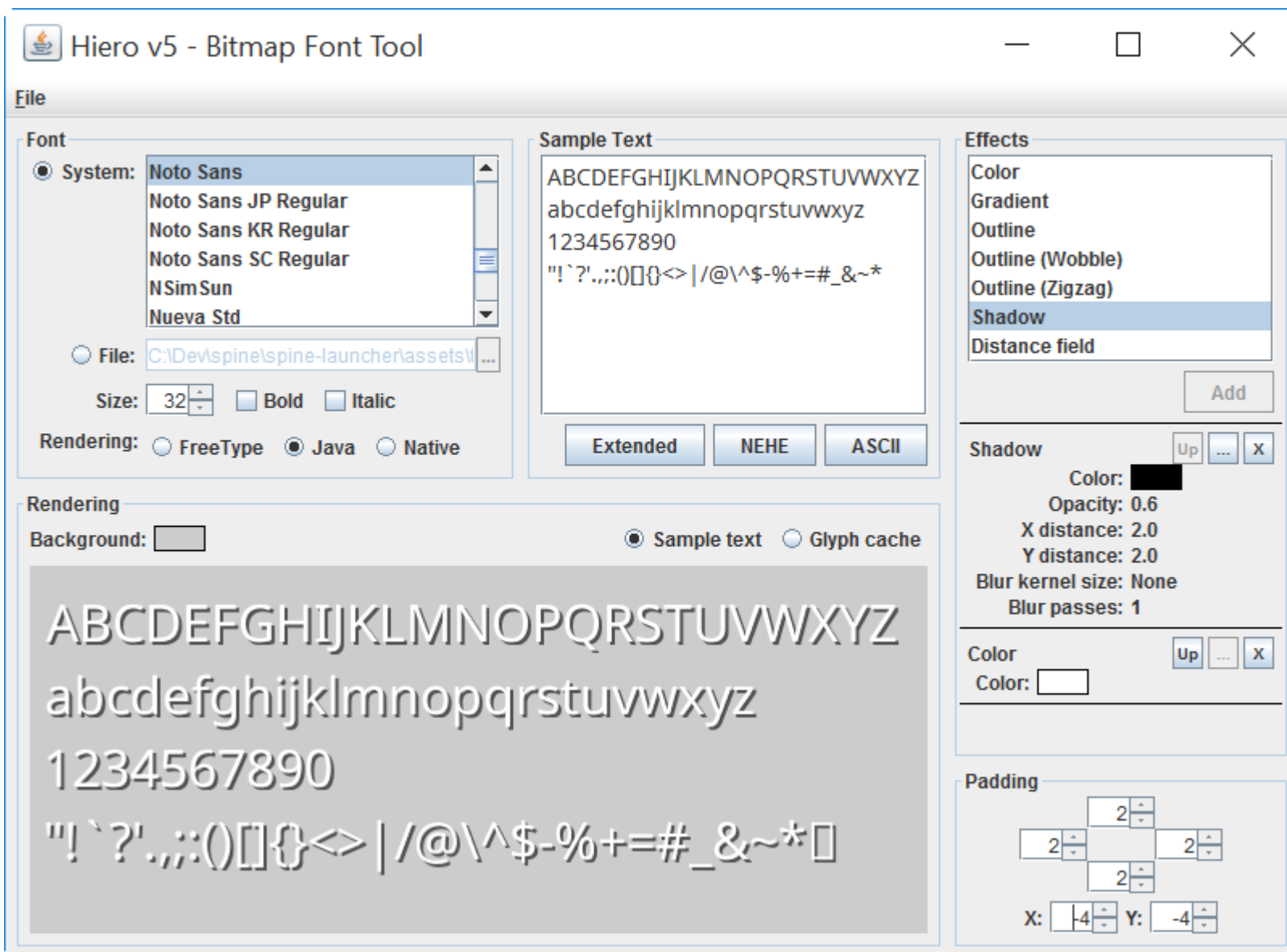
- Vocês devem ter reparado até agora com a experiência da libgdx que tudo que é “desenhado” na tela por controladores gráficos como o OpenGL **são texturas** (e a libgdx é **apenas um link ou framework que trabalha em cima do OpenGL**).
- Dito isto, como seria possível escrever texto nesse contexto?
 - Os textos também são desenhados a partir de texturas. As letras **estão em uma textura** e os caracteres são escritos a partir de um “texture atlas”.



BITMAP FONTS

- Existe também um programa da própria libgdx para **criar as fontes**, facilitando o trabalho.
- O nome desse programa é **hiero** e vocês podem baixá-lo no seguinte link:
 - <https://libgdx.badlogicgames.com/nightlies/runnables/runnable-hiero.jar>
- Basta selecionar a fonte, tamanho e parâmetros, adicionar efeitos desejados e depois ir em **File → Save BMFont files e salvar o arquivo**.
 - Na hora de carregar na libgdx vocês devem carregar o .fnt e .png gerados.

HIERO





BITMAP FONTS

- Basta instanciar uma nova fonte:

```
BitmapFont font = new BitmapFont(Gdx.files.internal("data/nomeFonte.fnt"),  
Gdx.files.internal("data/nomeFonte.png"), false);
```

- Ou

```
private static BitmapFontLoader.BitmapFontParameter bfp = new  
BitmapFontLoader.BitmapFontParameter();  
public static void loadBitmapFont(String path){  
    bfp.flip = true; //caso queira mudar algum parâmetro  
    assetManager.load(path, BitmapFont.class, bfp);  
}
```

- E depois

```
assetManager.get(path, BitmapFont.class);
```

ESCREVER USANDO AS BITMAP FONTS

- Para escrever um texto qualquer, basta utilizar os comandos abaixo:

```
font.draw(spriteBatch, "texto qualquer", posicaoX, posicaoY);
```

- Em certos momentos, é necessário saber o tamanho do texto (altura e largura) para poder centralizá-lo, dessa forma, podemos utilizar:

```
GlyphLayout glyphLayout = new GlyphLayout();  
glyphLayout.setText(font, Gdx.graphics.getFramesPerSecond() + " FPS");  
System.out.println(glyphLayout.width + "," + glyphLayout.height);
```


- # limitar a largura





LIMITAR LARGURA DA BITMAP FONT

- Para solucionar essa questão, também utilizamos o GlyphLayout:

```
layout.setText(font, currentMessage, font.getColor(), textboxWidth, Align.Left, true);
```

```
font.draw(batch, layout, x, y);
```

- Os parâmetros do método **setText** são, respectivamente:
 - BitmapFont
 - CharSequence
 - Color
 - Float (targetWidth)
 - Int (halign)
 - Boolean (wrap)

INTERPOLAÇÕES EM BITMAP FONTS

- Existe um problema com as **BitmapFonts** que é o **redimensionamento**. Redimensionar qualquer imagem (aumentar ou diminuir seu tamanho) **implica em interpolações**. As interpolações podem ser (normalmente):
 - Vizinho mais próximo (nearest neighbor)
 - Interpolação linear (linear interpolation)
 - Interpolação bilinear (bilinear interpolation)
 - Interpolação bicúbica (bicubic interpolation)
- Diferentes interpolações vão gerar **efeitos diferentes na imagem**.
 - Contudo, nenhuma das interpolações vai deixar a imagem tão boa quanto na resolução original. **Elas podem acabar mais desfocadas ou pixeladas de forma estranha**.

BITMAP FONTS

Original



Zoom pra fora



Zoom pra dentro





FREE TYPE FONTS

- Existe uma forma de gerar as fontes em “tempo real” ao invés de renderizá-las em bitmap com o Hiero.
 - A diferença é que numa font pré-renderizada é muito mais fácil adicionar efeito às letras. No caso do free type, você não tem essa opção.
- Mais detalhes podem ser encontrados em:
 - <https://libgdx.com/wiki/extensions/gdx-freetype>
- Não iremos utilizá-las aqui na disciplina pois a Bitmap Font é suficiente na extrema maioria das situações.



FREE TYPE FONTS

- Exemplo de utilização:

```
FreeTypeFontGenerator generator = new  
FreeTypeFontGenerator(Gdx.files.internal("data/unbom.ttf"));
```

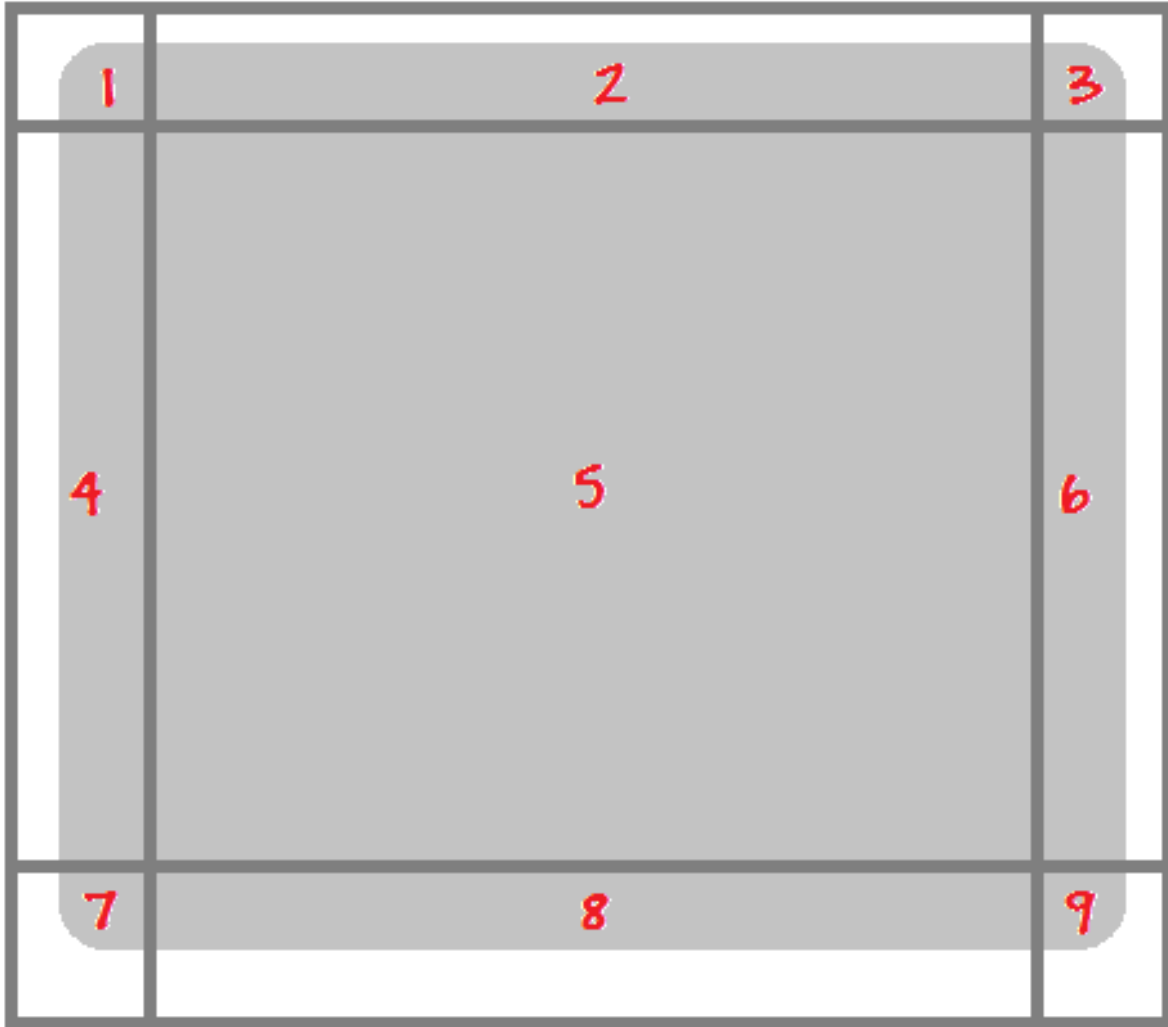
```
FreeTypeFontParameter parameter = new FreeTypeFontParameter();  
parameter.size = 18;  
parameter.characters = "한국어/조선";
```

```
BitmapFont koreanFont = generator.generateFont(parameter); //gera a bitmapfont aqui  
em tempo real
```

```
parameter.characters = FreeTypeFontGenerator.DEFAULT_CHARS;  
generator = new FreeTypeFontGenerator(Gdx.files.internal("data/russkij.ttf"));  
BitmapFont cyrillicFont = generator.generateFont(parameter);
```

```
generator.dispose(); //tem que fazer o dispose depois de gerar as fontes
```

NINE PATCH



Nine Patches





NINE PATCH

- Um **nine patch** ou **nine patches**, no plural, é uma imagem com uma área **esticável** definida. Uma vez que as áreas são predefinidas, não será visível qualquer distorção na imagem original, se feito da forma correta.
- Nine patches **são utilizados** em:
 - Botões
 - Caixas de texto (balões de fala, etc).
 - ScrollPanels
 - TextFields
 - etc



NINE PATCH

- Um ninepatch pode ser instanciado da seguinte forma:

```
NinePatch patch = new NinePatch(new Texture(Gdx.files.internal("knob.png")), 12,  
12, 12, 12);
```

- Os quatro inteiros (preenchido com 12 no caso) são os **espaçamentos nas bordas** seguindo a seguinte ordem:

- Esquerda
- Direita
- Cima
- Baixo

- Para desenhar:

```
patch.draw(batch, x, y, width, height);
```

EXERCÍCIO

- Faça uma bolha de diálogo utilizando a [BitmapFont](#) e [NinePatch](#).
 - Limite a largura da caixa de texto.
- O usuário deve ser capaz de digitar o texto e dar enter. Depois do enter a mensagem é escrita dentro da bolha de diálogo, [em cima do seu personagem](#).
- Vocês podem baixar um exemplo de ninepatch no seguinte código:

45U-6VY



BOX2D

- Existe uma biblioteca já pronta para tratamento de **colisão e emulação física** (gravidade, etc), chamada Box2D. Caso tenha interesse, pode ver um tutorial completo no seguinte link:
 - <https://www.codeandweb.com/texturepacker/tutorials/libgdx-physics>
- Exemplo em um video:
 - <https://www.youtube.com/watch?v=Dp4EVADZWHc>
- Contudo, não vamos utilizar o Box2D diretamente aqui na disciplina (embora você possa usar sem problemas caso queira).
- Dessa forma, trataremos as **colisões e hitboxes na mão**.



HIT BOXES

- Uma definição de hitbox é uma **área invisível**, normalmente um **retângulo**, em que existe uma checagem se houve uma interação nessa área. Caso exista essa interação, isso é contabilizado como um **hit/colisão**.
- Em caso de tiros, por exemplo, você verifica se a posição da **bala entrou dentro do hitbox**.
 - Em jogos de tiro muitas vezes existe uma hitbox para a cabeça e uma hitbox para o corpo.
 - No overwatch, por exemplo, a hitbox é **circular, parecendo diversos cilindros**.

HIT BOXES

- Exemplos no overwatch:



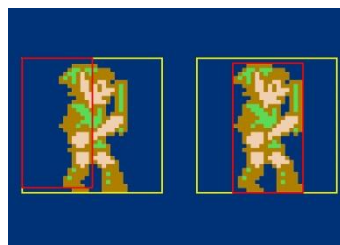
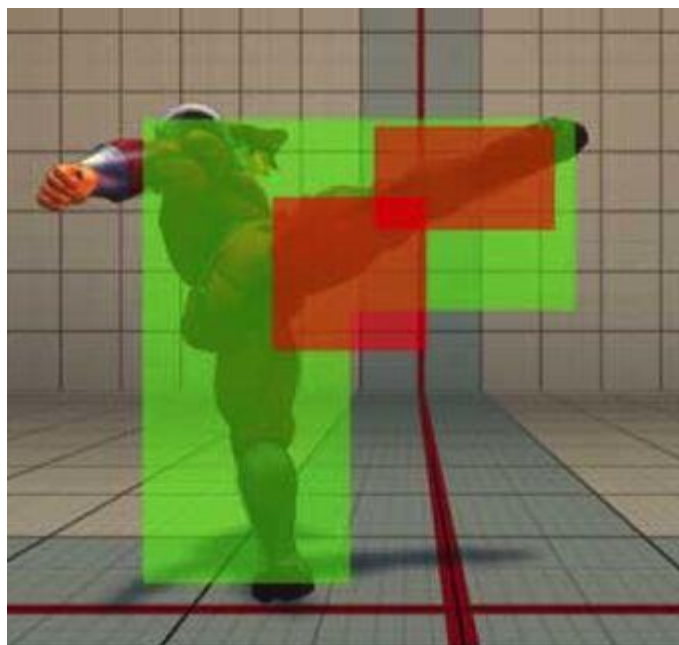
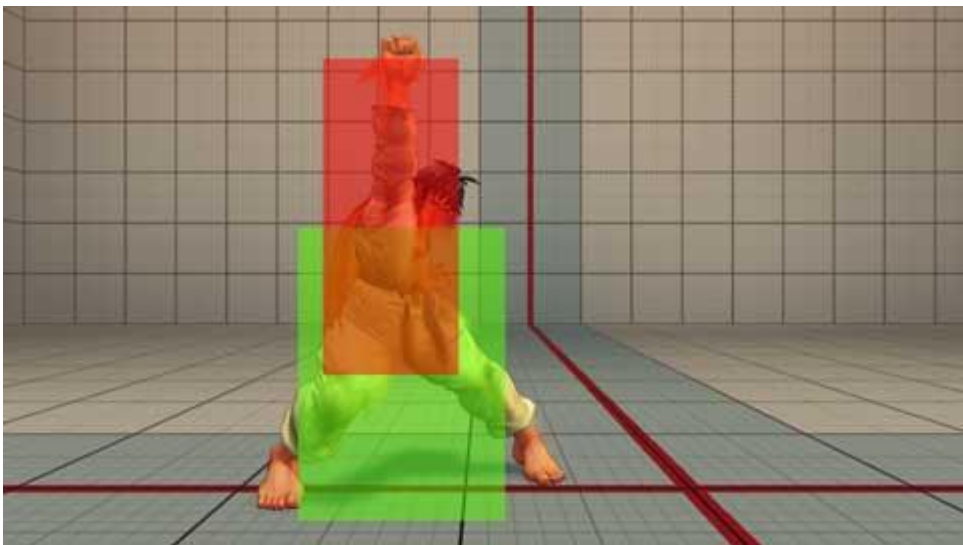


HIT BOXES

- Basicamente, em um jogo como o overwatch, **projéteis são atirados ao longo do mundo** (eles de fato viajam como uma bala no caso de vários personagens).
 - Exemplos de personagem que atiram projétil: hanzo, junkrat, mei, echo, genji, etc.
 - Caso esse projétil **entre dentro das áreas de hitboxes** ilustradas no slide anterior, isso é contabilizado como um acerto.
- Existem outros tiros como **hitscan** em que o acerto é instantâneo.
 - Basicamente, é como se um **raio de luz** (uma linha) fosse traçado no momento do tiro, e caso ele esteja dentro do hitbox, um acerto é contabilizado.
 - No caso do counter strike, por exemplo, todos os tiros são **hitscan**.

HIT BOXES

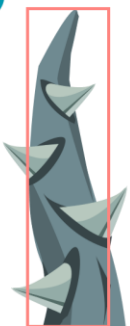
- Em “jogos 2d” o cálculo dos hitboxes é bastante fácil. Basta circular a imagem ou o objecto com um ou vários retângulos imaginários:



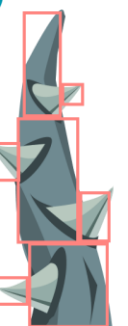
1



2

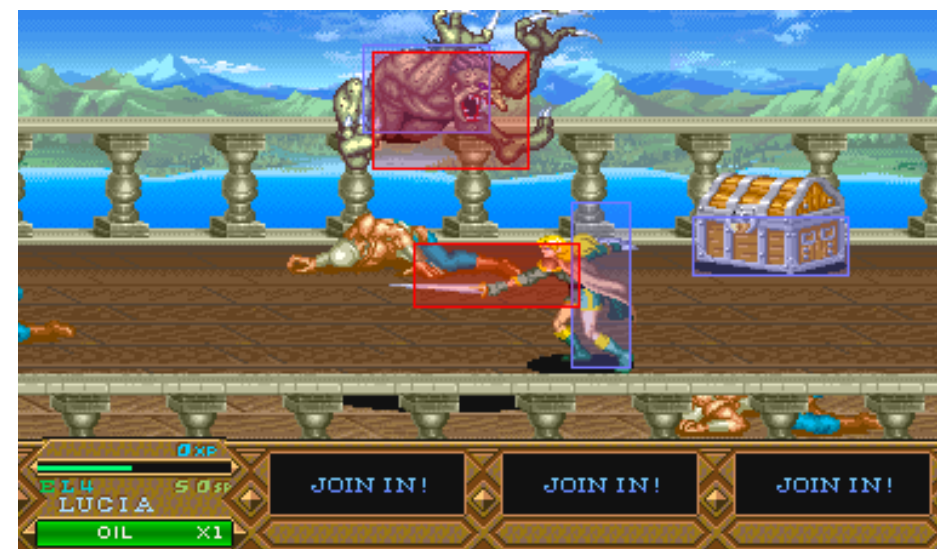
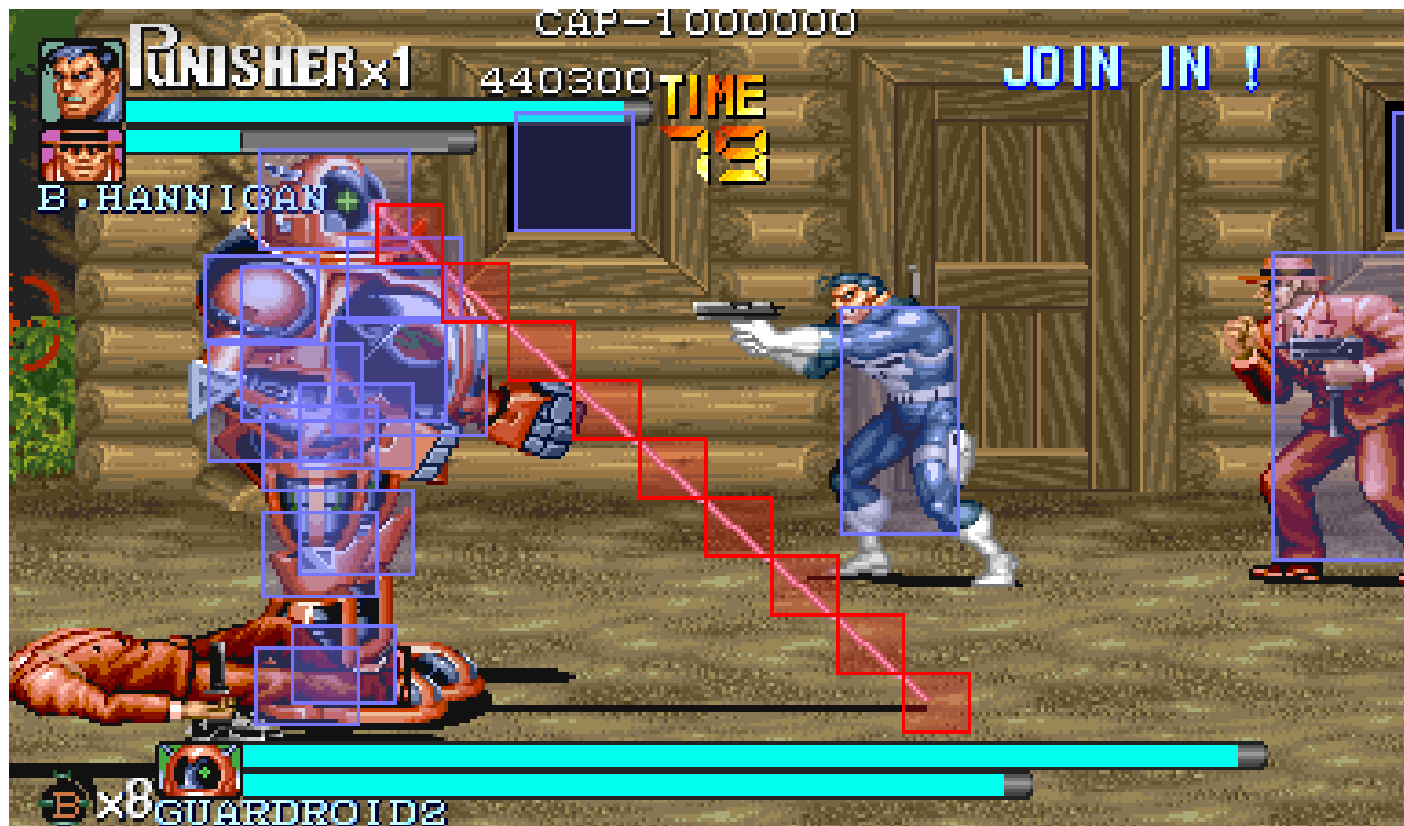


3



HIT BOXES

- Em um exemplo 2d todas as **balas ou tiros** contêm um hitbox e caso o seu hitbox entre dentro do hitbox de algum personagem, acontece um acerto.





HIT BOXES

- O retângulo é o hitbox **mais rápido de ser computado**, embora seja **menos preciso em vários casos**.
- Para saber um ponto está dentro do retângulo basta ver se **as coordenadas do ponto são maiores que o ponto do baixo-esquerda do retângulo e menores que o ponto topo-direita**.
- Já com dois retângulos basta olhar se os pontos das **bordas de um triângulo entraram dentro do outro triângulo** com a mesma lógica acima.

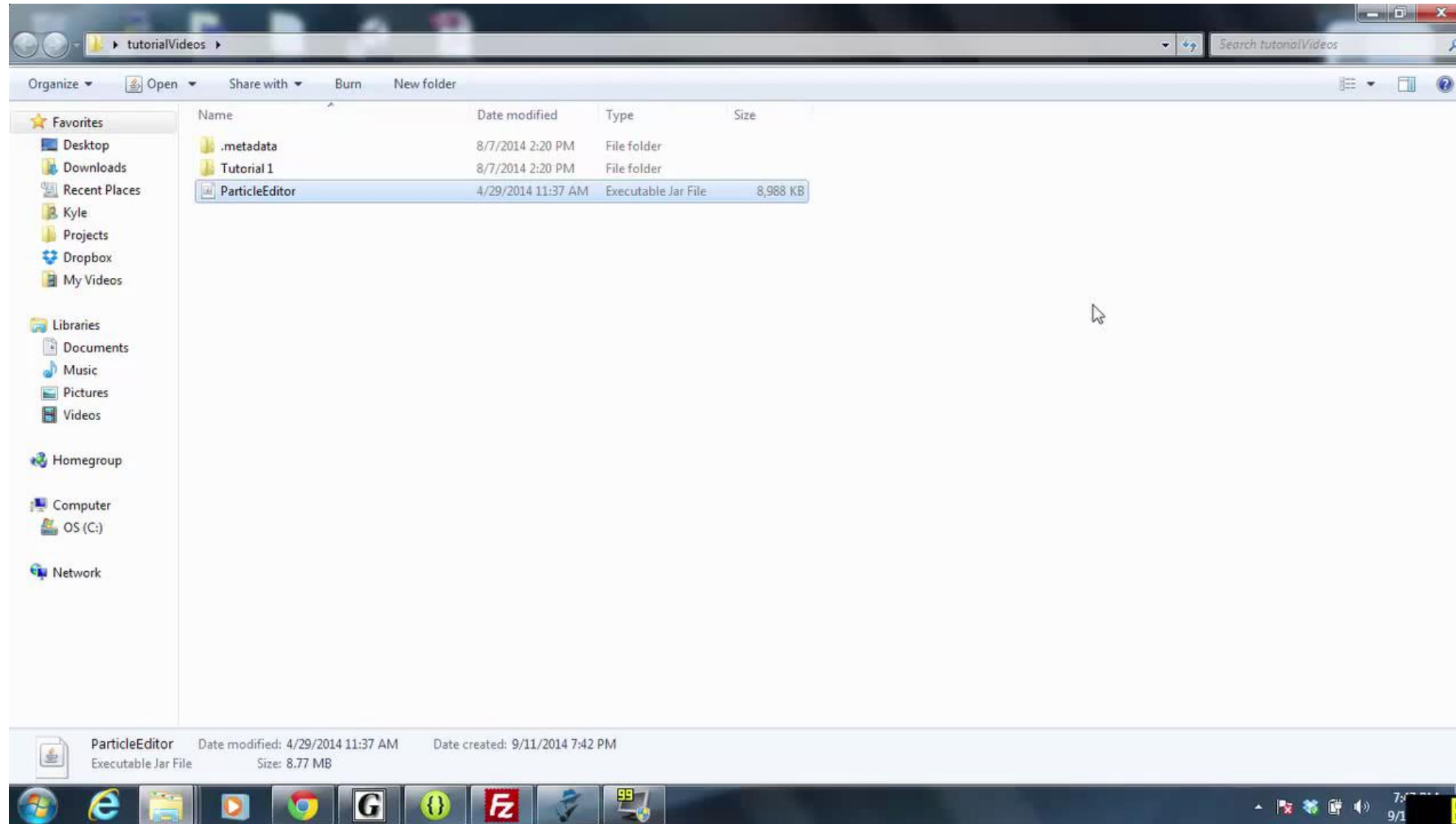
EXERCÍCIO

- Coloque hitboxes nas balas do seu jogo e nos personagens. Caso a hitbox da bala entre dentro da hitbox do personagem, apague o personagem da tela (ou colocar um dano subindo).
- Coloque uma opção “debug” no seu jogo de forma que quando clicamos no botão debug, ou ativamos o “modo debug” através de uma variável, você desenha no lugar dos hitboxes retângulos transparentes (desenha só a borda) usando o ShapeRenderer.

PARTÍCULAS 2D

- Efeitos de magia ou efeitos gráficos podem ser gerados através de partículas.
- Partículas são aceleradas pela GPU.
 - Elas se comportam como entidades independentes.
 - A ideia toda é bastante paralela (arquitetura da GPU).
- Assim como em outras situações, existe um programa da libgdx para gerar as partículas:
 - <https://libgdx.badlogicgames.com/nightlies/unnables/unnable-2D-particles.jar>

PARTICLE EDITOR



<https://www.youtube.com/watch?v=520vA2eIV4M>

CARREGAR AS PARTÍCULAS

- O código para abrir e executar uma partícula pode ser o seguinte:

```
TextureAtlas particleAtlas; //<-declara um atlas com a partícula
```

```
ParticleEffect effect = new ParticleEffect();  
effect.load(Gdx.files.internal("myparticle.p"), particleAtlas);  
effect.start();
```

```
//Setando a posição do efeito  
effect.setPosition(x, y);
```

```
//Atualizando e desenhando o efeito  
//Delta simboliza a passagem do tempo  
effect.draw(batch, delta);
```

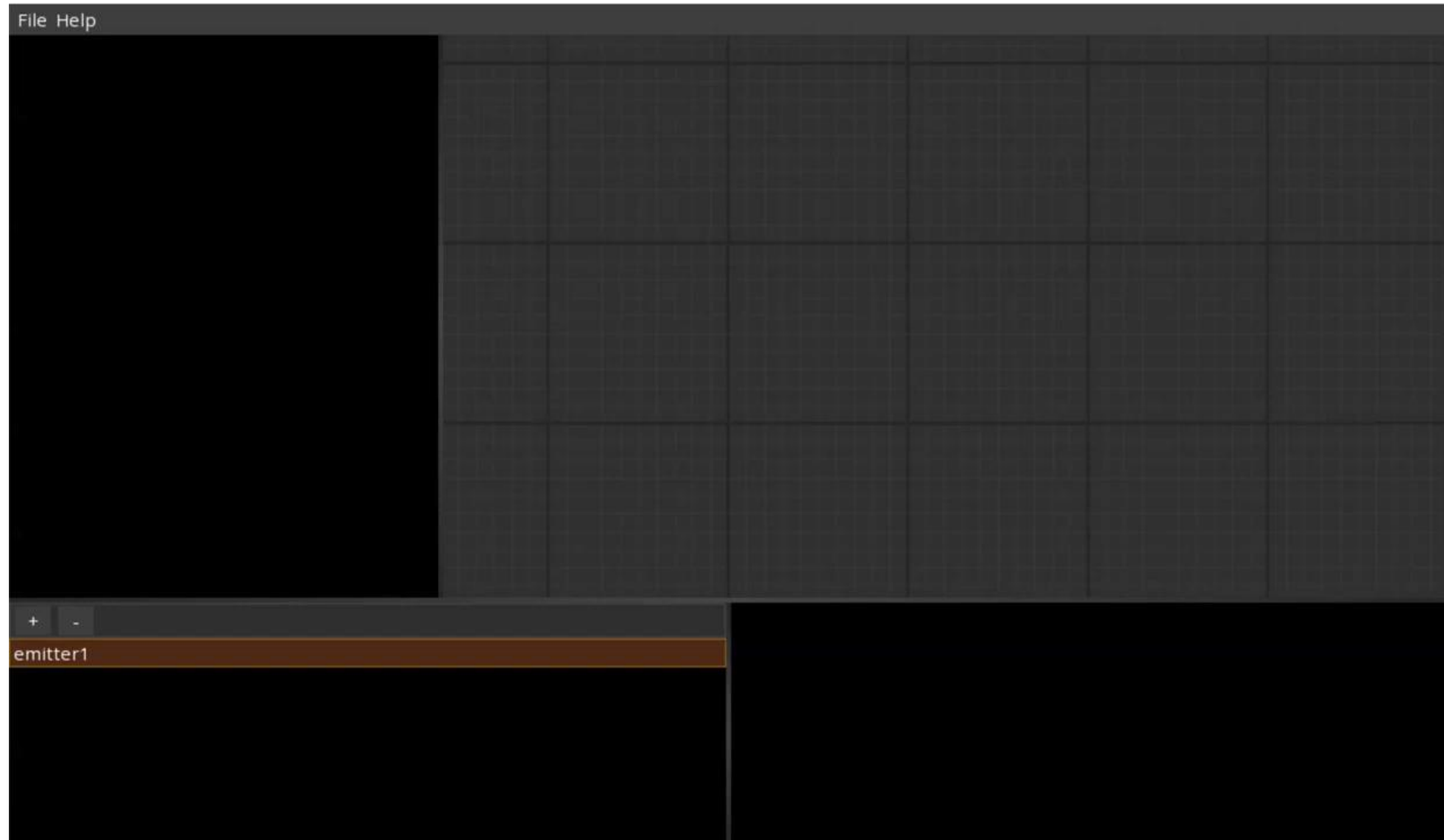


PARTÍCULAS 2D

- Exemplo de código que **emite partículas** usando partículas geradas pelo Particle Editor:
 - <https://github.com/libgdx/libgdx/blob/master/tests/gdx-tests/src/com/badlogic/gdx/tests/ParticleEmitterTest.java>



TALOS PARTICLE EDITOR



<https://www.youtube.com/watch?v=j8L8B68a9qw>



EXERCÍCIO

- Altere o código do seu jogo adicionando um efeito gráfico com partículas 2D quando o hitbox da bala atinge o alvo (personagem).
- Não esqueça de manter o som do efeito também quando houver o hit!